Context Dynamic Injection

Table of Contents

1. Overview: Layered Architecture of Context Injection
2. Layer 1: System Reminder Dynamic Injection
   - Core Mechanism
   - Injection Scenarios
   - Injection Content Classification
   - System Reminder Collection in Beta Tracing
   - Message Types Wrapped in `<system-reminder>`
3. Layer 2: CLAUDE.md / Memory File Injection
   - File Discovery Hierarchy
   - Key Functions
   - @ Directive Injection
   - HTML Comment Stripping
   - MEMORY.md Entrypoint Truncation
   - Conditional Rules
   - Integration with `context.ts`
   - Cache Invalidation Mechanism
4. Layer 3: Auto Memory System
   - Overview
   - Memory Type Classification
   - Persistence Structure
   - Memory File Frontmatter Format
   - Integration with System Prompt
   - Behavioral Constraint Guidelines
   - Stale Memory Verification Guidelines
5. Layer 4: Session Memory
   - Overview
   - Trigger Mechanism
   - Trigger Thresholds
   - Isolation Mechanism
   - Integration with Compaction
6. Layer 5: ExtractMemories Service
   - Overview
   - Trigger Mechanism
   - Mutual Exclusion
   - Tool Permission Control
   - Extraction Throttling
   - Trailing Run
7. Layer 6: Hooks System (Lifecycle Injection)

- Overview
- Hook Types (by Trigger Timing)
- InstructionsLoaded Hook (Conditional Injection)
- Hook Output Injection
- asyncRewake Mechanism (Background Hook Wakeup)
- Working Directory Trust Check

8. Layer 7: System Prompt Segmentation and Dynamic Boundaries
   - SYSTEM_PROMPT_DYNAMIC_BOUNDARY
   - Cache Breaker Injection
   - Cache Break Detection
9. Layer 8: Conditional Rules Injection
   - Nested Memory Loading Mechanism
   - Glob Pattern Matching
   - Base Path Resolution
10. Post-Sampling Hook
    - Core Mechanism
    - Usage Scenarios
11. appendSystemPrompt and overrideSystemPrompt
    - Two-Tier Extra Injection
    - Priority Hierarchy
    - Cowork Extra Guidelines Injection
12. Coordinator and Agent Prompt Priority
    - Coordinator Mode
    - Agent Definition Injection
13. Compact (Context Window Compression)
    - Compression Trigger Timing
    - Memory Handling During Compression
    - Compact Boundary Markers
14. Architecture Summary and Key Design Patterns
    - Layered Responsibility Overview
    - Key Design Patterns

---

1. Overview: Layered Architecture of Context Injection

Claude Code's dynamic context injection is not a single mechanism — it is composed of **13 independent subsystems**, each with clear responsibilities and well-defined boundaries, ultimately converging into API calls through a unified message construction pipeline. The overall architecture can be summarized as:

```
API call (query())
  ↑
  ┌─ systemPrompt (from prompts.ts / systemPromptSections.ts)
  ├─ userContext (from context.ts: getUserContext)
  ├─ systemContext (from context.ts: getSystemContext)
```

```
  ├─ messages[] (user messages + system attachment messages)
  └─ attachments (skills, agents, mcp, tool deltas, etc.)
```

---

2. Layer 1: System Reminder Dynamic Injection

2.1 Core Mechanism

System Reminder is Claude Code's most fundamental context injection mechanism. By wrapping system-level information in `<system-reminder>...</system-reminder>` XML tags, it **injects system-level information inside user messages**, making it part of the model's attention context.

Core function: `src/utils/messages.ts`

```
// Entry point
export function wrapInSystemReminder(content: string): string {
  return `<system-reminder>\n${content}\n</system-reminder>`
}

// Batch wrapping
export function wrapMessagesInSystemReminder(
  messages: Message[],
): Message[]
```

2.2 Injection Scenarios

All messages of type `system/init` constructed via `buildSystemInitMessage()`, as well as attachment messages constructed by various attachment builder functions, are automatically wrapped in `<system-reminder>` tags:

```
// src/utils/messages.ts - attachment wrapping example
return wrapMessagesInSystemReminder([
  createAttachmentMessage({
    type: 'system_local_command',
    content: `${formatShellPrefixCommand(command, workingDirectory)} ${input}`,
  }),
])
```

2.3 Injection Content Classification

The system prompt itself (`prompts.ts`) also includes documentation about System Reminders:

```
function getSystemRemindersSection(): string {
  return `- Tool results and user messages may include <system-reminder> tags.
    <system-reminder> tags contain useful information and reminders. They are
    automatically added by the system, and bear no direct relation to the specific
    tool results or user messages in which they appear.
  - The conversation has unlimited context through automatic summarization.`
}
```

2.4 System Reminder Collection in Beta Tracing

In `src/utils/telemetry/betaSessionTracing.ts`, the system extracts all system-reminder content from messages for telemetry:

```
function extractSystemReminderContent(text: string): string | null {
  const match = text.match(/<system-reminder>([\s\S]*?)<\/system-reminder>/)
  return match ? match[1] : null
}
```

2.5 Message Types Wrapped in `<system-reminder>`

All messages processed through `normalizeAttachmentForAPI()`, after being wrapped by `wrapMessagesInSystemReminder()`, carry `<system-reminder>` tags in two forms:

- String content: the entire content is wrapped as `<system-reminder>\n{content}\n</system-reminder>`
- Array content: only blocks with `type: 'text'` are wrapped; other block types like `type: 'tool_result'` remain unchanged

Content that is actually wrapped (attachments):

类型	内容说明
plan_mode / plan_mode_reentry / plan_mode_exit	计划模式工作流指令
auto_mode / auto_mode_exit	自动模式指令
edited_text_file	文件变更通知
compact_file_reference	摘要后旧文件引用
pdf_reference	大 PDF 轻量引用
selected_lines_in_ide	IDE 选中行
opened_file_in_ide	IDE 打开文件通知
plan_file_reference	计划文件内容
invoked_skills	已调用技能内容
todo_reminder	TodoWrite 提醒
task_reminder	TaskCreate/TaskUpdate 提醒
nested_memory / relevant_memories	记忆内容
skill_listing / skill_discovery	技能列表/发现
queued_command	本地排队命令
output_style	输出风格通知
diagnostics	诊断问题
mcp_resource	MCP 资源内容
agent_mention	Agent 提及
task_status	后台任务状态
async_hook_response	Hook 异步响应
token_usage / budget_usd / output_token_usage	限额统计
hook_blocking_error / hook_success / hook_additional_context / hook_stopped_continuation	Hook 事件
compaction_reminder / context_efficiency	自动压缩提示
date_change	日期变更
ultrathink_effort	推理 effort 级别
deferred_tools_delta / agent_listing_delta / mcp_instructions_delta	增量更新通知
companion_intro	Companion 介绍
verify_plan_reminder	VerifyPlanExecution 调用提醒
critical_system_reminder	任意关键提醒
teammate_mailbox / team_context	Swarm 多智能体消息

Design Philosophy: These entries are all information "the model needs to act upon, but should not directly tell the user."

They are categorized as follows:

Directives that change model behavior
`plan_mode`, `auto_mode`, `verify_plan_reminder`, `ultrathink_effort` — the user has switched to a certain working mode, and the system needs to inform the model "how you should behave now"

System state changes known to the user
`edited_text_file`, `selected_lines_in_ide`, `opened_file_in_ide`, `date_change` — the user edited the file or selected the lines themselves; the model needs to know these events happened, but should not say in its reply "I notice you edited the file"

Background knowledge that tools cannot proactively fetch
`invoked_skills`, `relevant_memories`, `nested_memory`, `mcp_resource` — the user did not request this information via tools; it is context actively injected by the system

System-internal state / warnings
`diagnostics`, `compaction_reminder`, `context_efficiency`, `token_usage`, `budget_usd` — fully internalized system signals; even if the model knows them, it should not inform the user, nor should it explain them to the user

Hook event streams
`hook_blocking_error`, `hook_success`, `hook_additional_context`, etc. — feedback on hook execution results, for the model to make decisions

---

3. Layer 2: CLAUDE.md / Memory File Injection

3.1 File Discovery Hierarchy

`src/utils/claudemd.ts` implements a complete CLAUDE.md file discovery and injection pipeline. Files are loaded in the following priority order (lower priority first, higher priority later; the model pays more attention to content loaded later):

```
Loading order (low to high):
1. Managed (global managed level, /etc/claude-code/CLAUDE.md)
2. User (user global level, ~/.claude/CLAUDE.md)
3. Project (project level, CLAUDE.md / .claude/CLAUDE.md / .claude/rules/*.md found by
traversing upward)
4. Local (local level, CLAUDE.local.md)

Final merge into a single string, concatenated segment by segment in the form "Contents of
<path>:"
```

3.2 Key Functions

```
// Main entry point: discover and load all memory files
export const getMemoryFiles = memoize(
  async (forceIncludeExternal: boolean = false): Promise<MemoryFileInfo[]>
)
```

```
// Convert file list to system prompt string
export const getClaudeMds = (memoryFiles: MemoryFileInfo[]): string => {
  // Add descriptive suffix for each type, formatted as:
  // "Contents of /path/to/CLAUDE.md (project instructions, checked into the
codebase):\n\n<content>"
}
```

3.3 @ Directive Injection

Memory files support including other files via `@path` syntax:

```
// @include supported path formats
// @path, @./relative/path, @~/home/path, @/absolute/path
// Only expanded in plain text nodes; not expanded inside code blocks or HTML comments
// Maximum recursion depth: 5 levels
```

Note: `@path` references in memory files are expanded at load time, injecting the target file's content

3.4 HTML Comment Stripping

When parsing memory file content, block-level HTML comments (`<!-- ... -->`) are automatically stripped:

```
// src/utils/claudemd.ts
export function stripHtmlComments(content: string): { content: string; stripped: boolean }
```

Note:

- HTML comments manually written in CLAUDE.md are essentially human-only annotations
- This portion corresponds to content the author wrote in CLAUDE.md but does not want the AI to see — essentially **annotations visible to humans but hidden from the AI**

Typical scenarios:

```
  1. Internal TODO / action item annotations
  <!-- TODO: this rule needs to be rewritten after v2 release -->
  <!-- FIXME: temporary workaround, remove this rule in production -->
  Please strictly follow these API specifications...

  2. Draft content (not yet finalized)
  <!-- The specifications below are still under discussion, not yet in effect -->
  ## Experimental Feature Handling
  Confirming with the team...

  3. Instructions only needed by certain people
  <!-- Hint for maintainers: this directory structure will be refactored next version -->
  ## Deployment Configuration

  4. Context comments for multi-person collaboration
  <!-- 2024-06 Added this rule after security review -->
  Do not directly concatenate user input into SQL.

  <!-- legal-team-2024-07: This exception clause was added at legal's request -->

  5. Conditional comments (debug / development only)
```

```
  <!-- DEBUG: skip certificate verification in local development -->
  When connecting to external services, verify certificates...
```

Design Intent

The author wants to preserve these human collaboration traces (review records, TODOs, draft content) in CLAUDE.md, but does not want them to enter the AI context — because:

- TODOs/drafts can confuse the AI
- Internal analysis comments pollute the context
- Temporary workarounds should not influence the AI's judgment

The stripping logic's design: **comments written in block-level form (occupying their own line) are by default assumed to be human-only annotations**; if written inline (like `Use <method>`), it indicates the author is quoting the comment syntax in the body text, which is meaningful content and should not be stripped

3.5 MEMORY.md Entrypoint Truncation

MEMORY.md has dual limits:

```
// src/memdir/memdir.ts
export const MAX_ENTRYPOINT_LINES = 200   // Max 200 lines
export const MAX_ENTRYPOINT_BYTES = 25_000 // Max 25KB

// Warning appended when truncated:
"> WARNING: MEMORY.md is {reason}. Only part of it was loaded..."
```

Note: MEMORY.md is the memory index; it only stores pointers, formatted as `- [Title](filename.md) — short description`

3.6 Conditional Rules

Files can restrict their applicable scope via the frontmatter `paths` field:

```
---
paths:
  - src/**/*.ts
  - src/**/*.tsx
---
Your instructions here...
```

```
// src/utils/claudemd.ts
export async function processConditionedMdRules(
  targetPath: string,
  rulesDir: string,
  type: MemoryType,
  processedPaths: Set<string>,
  includeExternal: boolean,
): Promise<MemoryFileInfo[]>
// Uses the `ignore` library for glob pattern matching
```

The system matches the current working directory path against `glob` patterns, and only injects the memory when it matches.

Note:
This is a typical **context-aware** or **dynamic prompt injection** design. Simply put:

- Regular memories are "globally effective" — once loaded, they always exist in the context
- Conditional rules allow a memory file to declare: **"This memory is only useful when I'm editing files at specific paths, so only inject it when there's a match"**

3.7 Integration with `context.ts`

`getUserContext()` in `src/context.ts` calls `getClaudeMds()`:

```
// src/context.ts
export const getUserContext = memoize(async (): Promise<{ [k: string]: string }> => {
  const claudeMd = getClaudeMds(filterInjectedMemoryFiles(await getMemoryFiles()))
  setCachedClaudeMdContent(claudeMd || null) // Cached for use by other modules
  return {
    ...(claudeMd && { claudeMd }),
    currentDate: `Today's date is ${getLocalISODate()}.`,
  }
})
```

3.8 Cache Invalidation Mechanism

```
// Reset cache (triggers full reload)
export function resetGetMemoryFilesCache(reason: InstructionsLoadReason): void

// Clear cache only (no hook trigger; used for correctness purposes only)
export function clearMemoryFileCaches(): void

// loadReason propagation: 'session_start' | 'compact' | 'include'
let nextEagerLoadReason: InstructionsLoadReason = 'session_start'
```

Note: These two functions serve **hook notification + cache invalidation coordination**. Their relationship to dynamic injection is: **letting hook callers know why memory was reloaded**, so they can decide how to respond (e.g., after compaction the context needs to be rebuilt, while a worktree switch only requires a silent cache refresh)

---

4. Layer 3: Auto Memory System

Note: See the "Memory System" documentation for details

4.1 Overview

Auto Memory is a complete file-based persistent memory system that injects guiding prompts into the System Prompt via `loadMemoryPrompt()`. Core file: `src/memdir/memdir.ts`.

4.2 Memory Type Classification

The system enforces four memory types (`src/memdir/memoryTypes.ts`):

```
export const MEMORY_TYPES = ['user', 'feedback', 'project', 'reference'] as const
```

Each type has explicit `when_to_save`, `how_to_use`, and `body_structure` specifications.

4.3 Persistence Structure

```
~/.claude/projects/<slug>/memory/
  MEMORY.md          # Entrypoint index (one link per record)
  user_role.md       # Topic files (frontmatter + content)
  feedback_*.md
  project_*.md
  reference_*.md
```

4.4 Memory File Frontmatter Format

```
---
name: {{memory name}}
description: {{one-line description}}
type: {{user, feedback, project, reference}}
---
{{memory content}}
```

4.5 Integration with System Prompt

```
// src/memdir/memdir.ts
export async function loadMemoryPrompt(): Promise<string | null> {
  // 1. KAIROS daily log mode (append-only)
  if (feature('KAIROS') && autoEnabled && getKairosActive()) {
    return buildAssistantDailyLogPrompt(skipIndex)
  }
  // 2. TEAMMEM team memory mode
  if (feature('TEAMMEM') && teamMemPaths!.isTeamMemoryEnabled()) {
    return teamMemPrompts!.buildCombinedMemoryPrompt(extraGuidelines, skipIndex)
  }
  // 3. Standard auto memory
  if (autoEnabled) {
    return buildMemoryLines('auto memory', autoDir, extraGuidelines, skipIndex).join('\n')
  }
  return null
}
```

4.6 Behavioral Constraint Guidelines

The System Prompt injected by Auto Memory includes a detailed "should NOT save" list:

```
// src/memdir/memoryTypes.ts
export const WHAT_NOT_TO_SAVE_SECTION: readonly string[] = [
  '## What NOT to save in memory',
  '- Code patterns, conventions, architecture, file paths, or project structure...',
  '- Git history, recent changes, or who-changed-what...',
  '- Debugging solutions or fix recipes...',
  '- Anything already documented in CLAUDE.md files.',
  '- Ephemeral task details...',
  // Important: filter even when the user explicitly asks to save
```

```
  'These exclusions apply even when the user explicitly asks you to save.'
]
```

4.7 Stale Memory Verification Guidelines

```
// src/memdir/memoryTypes.ts
export const TRUSTING_RECALL_SECTION: readonly string[] = [
  '## Before recommending from memory',
  'A memory that names a specific function, file, or flag is a claim that it existed...',
  '- If the memory names a file path: check the file exists.',
  '- If the memory names a function or flag: grep for it.',
  '"The memory says X exists" is not the same as "X exists now."'
]
```

---

5. Layer 4: Session Memory

Note: See the "Context Management" documentation for details

5.1 Overview

Session Memory is a **session-level** memory mechanism that automatically maintains a Markdown file during conversation runtime, recording key information from the current session. Core file: `src/services/SessionMemory/sessionMemory.ts`.

5.2 Trigger Mechanism

Session Memory is triggered via **Post-Sampling Hook** (see Layer 9), running automatically after model sampling:

```
// src/services/SessionMemory/sessionMemory.ts
export async function extractSessionMemory(context: REPLHookContext): Promise<void> {
  // Only runs on the main REPL thread
  if (querySource !== 'repl_main_thread') return
  // Executes via forked subagent; does not block the main conversation
  await runForkedAgent({ querySource: 'session_memory', ... })
}
```

5.3 Trigger Thresholds

```
// Initialization threshold (how far into the conversation before first extraction)
minimumMessageTokensToInit: 1500  // token count

// Update threshold (minimum token growth between extractions)
minimumTokensBetweenUpdate: 3000

// Tool call threshold (number of tool calls since last extraction)
toolCallsBetweenUpdates: 50
```

5.4 Isolation Mechanism

Session Memory uses `runForkedAgent` to create an isolated context, preventing the child process from polluting parent process state:

```
// Isolate readFileState
overrides: { readFileState: setupContext.readFileState }
// Restrict tools: only allow editing specific memory files
canUseTool: createMemoryFileCanUseTool(memoryPath)
```

5.5 Integration with Compaction

```
// src/services/compact/sessionMemoryCompact.ts
// When context compression occurs, session memory content is included in the summarization
process
```

---

6. Layer 5: ExtractMemories Service

Note: See the "Memory System" documentation for details

6.1 Overview

The ExtractMemories service automatically extracts worth-persisting information from conversation history after each dialogue turn (stopHooks), writing it to the Auto Memory directory. Core file: `src/services/extractMemories/extractMemories.ts`.

6.2 Trigger Mechanism

```
// Called via handleStopHooks in stopHooks.ts
export async function executeExtractMemories(
  context: REPLHookContext,
  appendSystemMessage?: AppendSystemMessageFn,
): Promise<void>
```

6.3 Mutual Exclusion

Mutual exclusion strategy between ExtractMemories and the main Agent:

```
// If the main Agent just wrote a memory file, skip this extraction
if (hasMemoryWritesSince(messages, lastMemoryMessageUuid)) {
  return  // Skip to avoid duplication
}
```

6.4 Tool Permission Control

```
// src/services/extractMemories/extractMemories.ts
export function createAutoMemCanUseTool(memoryDir: string): CanUseToolFn {
  // Allowed: Read / Grep / Glob (unrestricted)
  // Allowed: Read-only Bash commands (ls, find, grep, cat, stat...)
  // Allowed: Edit / Write only for files within auto-memory directory
  // Denied: All other tools
}
```

6.5 Extraction Throttling

```
// Controlled via tengu_bramble_lintel GrowthBook config
// Default: execute extraction once every 1 turn
turnsSinceLastExtraction < getFeatureValue_CACHED_MAY_BE_STALE('tengu_bramble_lintel', 1)
```

6.6 Trailing Run

If new messages arrive during extraction, the latest context is stashed, and a trailing extraction is executed immediately after completion:

```
if (inProgress) {
  pendingContext = { context, appendSystemMessage }
  // Execute trailing run after current run completes
}
```

---

7. Layer 6: Hooks System (Lifecycle Injection)

Note: See the "Hook System Implementation" documentation for details

7.1 Overview

Hooks are shell commands or function callbacks configured by users in `.claude/settings.json`. They trigger at specific event points in Claude Code's lifecycle, and their results (JSON output) are injected back into the conversation context. Core file: `src/utils/hooks.ts`.

7.2 Hook Types (by Trigger Timing)

```
type HookEvent =
  | 'PreToolUse'           // Before tool call
  | 'PostToolUse'          // After tool call
  | 'PostToolUseFailure'   // After tool call failure
  | 'PreCompact'           // Before compaction
  | 'PostCompact'          // After compaction
  | 'Stop'                 // When conversation stops
  | 'StopFailure'          // When conversation abnormally stops
  | 'PreExecute'           // Before user submission
  | 'UserPromptSubmit'     // When user submits
  | 'PermissionsChange'    // When permissions change
  | 'SessionStart'         // When session starts
  | 'SessionEnd'           // When session ends
  | 'SubagentStart'        // When subagent starts
  | 'SubagentStop'         // When subagent stops
  | 'TaskCreated'          // When task is created
  | 'TaskCompleted'        // When task is completed
  | 'ConfigChange'         // When config changes
  | 'InstructionsLoaded'   // When instruction files load ← important!
  | 'Elicitation'          // Before user selection
  | 'ElicitationResult'   // After user selection
  | 'ToolResult'           // When tool result is processed
```

7.3 InstructionsLoaded Hook (Conditional Injection)

This is the key Hook for implementing "inject dedicated memory upon entering a certain path":

```
// Defined in src/utils/hooks.ts
interface InstructionsLoadedHookInput {
  path: string           // Path of the loaded file
  type: MemoryType       // File type: 'User' | 'Project' | 'Local' | 'Managed'
  load_reason: string    // Load reason: 'session_start' | 'compact' | 'include'
  globs?: string[]       // Path patterns from frontmatter
  parentFilePath?: string // Parent file path when included via @include
}

// Triggered in claudemd.ts
void executeInstructionsLoadedHooks(file.path, file.type, loadReason, {
  globs: file.globs,
  parentFilePath: file.parent,
})
```

7.4 Hook Output Injection

Hook-returned JSON output is injected via the following:

```
// systemMessage / additionalContext fields in HookResult
interface HookResult {
  systemMessage?: string        // Injected as system message
  additionalContext?: string   // Appended to context
  blockingError?: string       // Blocks operation
  initialUserMessage?: string  // Replaces user message
  updatedInput?: object        // Modifies tool input
  // ...
}
```

7.5 asyncRewake Mechanism (Background Hook Wakeup)

Async hooks can run in the background, injecting results into the conversation via `queued_command` attachments:

```
// src/utils/hooks.ts
void shellCommand.result.then(async result => {
  if (result.code === 2) {
    // exit code 2 = blocking error, injected as task-notification
    enqueuePendingNotification({
      value: wrapInSystemReminder(`Stop hook blocking error: ${stderr || stdout}`),
      mode: 'task-notification',
    })
  }
})
```

7.6 Working Directory Trust Check

```
// src/utils/hooks.ts
export function shouldSkipHookDueToTrust(): boolean {
  // Non-interactive mode (SDK) skips trust check
  const isInteractive = !getIsNonInteractiveSession()
  if (!isInteractive) return false
```

```
  // Interactive mode: all hooks require accepting trust dialog before execution
  const hasTrust = checkHasTrustDialogAccepted()
  return !hasTrust
}
```

---

8. Layer 7: System Prompt Segmentation and Dynamic Boundaries

8.1 SYSTEM_PROMPT_DYNAMIC_BOUNDARY

`src/constants/system.ts` defines a key boundary marker:

```
// src/constants/prompts.ts
export const SYSTEM_PROMPT_DYNAMIC_BOUNDARY =
  '__SYSTEM_PROMPT_DYNAMIC_BOUNDARY__'
```

Purpose: Marks the dividing line between static content (cacheable across organizations) and dynamic content (user/session-specific) in the System Prompt. Content before the boundary uses `scope: 'global'` caching; content after the boundary cannot be cached.

8.2 Cache Breaker Injection

`src/context.ts` contains a system prompt injection mechanism for cache break debugging:

```
// src/context.ts
let systemPromptInjection: string | null = null

export function setSystemPromptInjection(value: string | null): void {
  systemPromptInjection = value
  // Clear context cache
  getUserContext.cache.clear?.()
  getSystemContext.cache.clear?.()
}

// In getSystemContext:
if (feature('BREAK_CACHE_COMMAND')) {
  injection = getSystemPromptInjection()
  // Injected as cacheBreaker
  return { cacheBreaker: `[CACHE_BREAKER: ${injection}]` }
}
```

Note:

- cacheBreaker is a controllable cache break switch, **for internal debugging only** — by injecting a unique identifier in the system context of each request, engineers can precisely control when to break the Prompt Cache, enabling measurement and analysis of cache hit behavior

8.3 Cache Break Detection

`src/services/api/promptCacheBreakDetection.ts` monitors whether the prompt cache is being used correctly.

---

9. Layer 8: Conditional Rules Injection

9.1 Nested Memory Loading Mechanism

CLAUDE.md rules support two loading strategies:

1. **Eager Loading**: All rules are loaded at once at session start
2. **Nested/Demand Loading**: Conditional rules are loaded only when specific files/directories are accessed → the conditional-trigger-by-path approach for frontmatter in memories

```
// src/utils/claudemd.ts
// Phase 1: getManagedAndUserConditionalRules - load global conditional rules
export async function getManagedAndUserConditionalRules(
  targetPath: string,
  processedPaths: Set<string>,
): Promise<MemoryFileInfo[]>

// Phase 2: getMemoryFilesForNestedDirectory - load nested directory rules
export async function getMemoryFilesForNestedDirectory(
  dir: string,
  targetPath: string,
  processedPaths: Set<string>,
): Promise<MemoryFileInfo[]>

// Phase 3: getConditionalRulesForCwdLevelDirectory - load CWD-level rules
export async function getConditionalRulesForCwdLevelDirectory(...)
```

9.2 Glob Pattern Matching

```
// src/utils/claudemd.ts
// Uses the `ignore` library (respects .gitignore semantics)
import ignore from 'ignore'
return ignore().add(file.globs).ignores(relativePath)

// Supported frontmatter format:
paths:
  - "src/**/*.ts"
  - "src/**/*.tsx"
```

9.3 Base Path Resolution

Path globs have different base directories depending on the rule type:

```
const baseDir =
  type === 'Project'
    ? dirname(dirname(rulesDir)) // Parent of .claude/rules/ = project root
    : getOriginalCwd()           // Managed/User rules relative to original CWD
```

Claude Code 的规则文件（.md）有三个来源，对应三个 type：

type	规则文件目录	典型路径
Managed	~/.claude/rules/	全局配置目录，存放统一管理的规则
User	~/.claude/rules/	用户个人目录
Project	<项目根>/.claude/rules/	随项目代码库一起版本控制的规则

- **Project type**: `rulesDir = /path/to/project/.claude/rules`, going up two levels gives `/path/to/project` — the project root. Project rules are written by team members in CLAUDE.md or .claude/rules/. The rule author's mental reference when describing glob patterns is "project root", so `globs: ['src/**/*.ts']` is a path relative to the project root.
- **Managed/User type**: These rules reside under `~/.claude/rules/`. Users may run Claude in any directory. When the rule author writes a glob, the "current directory" they reference should be the directory where the user started Claude (i.e., `getOriginalCwd()`), not the home directory. So `globs: ['project/src/**/*.ts']` is interpreted as "relative to where I ran Claude".

---

10. Post-Sampling Hook

Note:
"Sampling" is Anthropic API terminology referring to **the process of the model generating a token from a probability distribution**. A complete "sample" is the model generating a full response — an assistant message containing all content blocks (text, tool_use, etc.)

```
Post-Sampling Hook 的时机

发送请求 → API 返回流 → 模型完成采样（回复生成完毕）
                                ↓
                        executePostSamplingHooks() ← 在这里执行
                                ↓
                        执行工具（tool_use 触发 CCR）
                                ↓
                        返回结果给模型（继续下一轮）
```

Timing: The model has completely generated a response but has not yet started executing tools
Purpose: The core value of this Hook is using the model's just-generated content for post-processing, without blocking the tool execution chain (performing additional side effects/convenient tasks based on model output)

10.1 Core Mechanism

`src/utils/hooks/postSamplingHooks.ts` defines an internal API that allows executing side effects after model sampling:

```
// Post-sampling hook registry (internal API, not exposed in settings.json)
const postSamplingHooks: PostSamplingHook[] = []

export function registerPostSamplingHook(hook: PostSamplingHook): void {
  postSamplingHooks.push(hook)
}

export async function executePostSamplingHooks(
  messages: Message[],
  systemPrompt: SystemPrompt,
  userContext: { [k: string]: string },
  systemContext: { [k: string]: string },
  toolUseContext: ToolUseContext,
  querySource?: QuerySource,
): Promise<void>
```

10.2 Usage Scenarios

Currently primarily used for Session Memory automatic extraction:

```ts
// src/services/SessionMemory/sessionMemory.ts
const extractSessionMemory = sequential(async function (context: REPLHookContext) {
  // Only runs on main thread
  if (context.querySource !== 'repl_main_thread') return
  // Check GrowthBook gate (non-blocking, cached value)
  if (!isSessionMemoryGateEnabled()) return
  // Check token and tool call thresholds
  if (!shouldExtractMemory(messages)) return
  // Fork subagent to execute extraction
  await runForkedAgent({ querySource: 'session_memory', ... })
})

export function initSessionMemory(): void {
  registerPostSamplingHook(extractSessionMemory)
}
```

Existing registered items:

Hook	作用
extractSessionMemory	从模型输出中提取值得记忆的内容，写入 auto memory
updateMagicDocs	如果模型改动了 MagicDocs 对应的文件，更新文档缓存
createSkillImprovementHook	分析 skill 执行结果，改进 skill 质量
Prompt 推测 (speculation)	基于模型输出预判下一步可能用到的提示
/auto-dream	生成梦境记忆

Distinction from other hooks:

- Pre-Sampling Hooks (before-tool-call, etc.): run before/after tool execution; can modify request content or intercept tools
- Post-Sampling Hook: runs after model response is generated, before tool execution; used for side-effect analysis
- Stop Hook: runs when model stops mid-generation; can inject additional context to let it continue

---

11. appendSystemPrompt and overrideSystemPrompt

11.1 Two-Tier Extra Injection

`buildEffectiveSystemPrompt()` in `src/utils/systemPrompt.ts` accepts two extra prompt parameters:

```ts
export function buildEffectiveSystemPrompt({
  overrideSystemPrompt,  // Priority 0: completely replaces all other prompts
  customSystemPrompt,     // Priority 3: custom prompt (via --system-prompt)
  appendSystemPrompt,     // Always appended at the end
  defaultSystemPrompt,
  // ...
}): SystemPrompt
```

11.2 Priority Hierarchy

```
Priority 0: overrideSystemPrompt → complete replacement, returns
asSystemPrompt([overrideSystemPrompt])
```

```
Priority 1: Coordinator mode → uses coordinator prompt

Priority 2: Agent mode (mainThreadAgentDefinition)
  - PROACTIVE/KAIROS mode: agent prompt appended to default
  - Other modes: agent prompt replaces default

Priority 3: customSystemPrompt (--system-prompt CLI flag)

Priority 4: defaultSystemPrompt

Priority 5: appendSystemPrompt (always appended at the end, except with
overrideSystemPrompt)
```

Note: **KAIROS/PROACTIVE Mode**:

- KAIROS/PROACTIVE transforms Claude Code from a "tool that waits to be called" into a "continuously running, proactive reporting assistant", achieved through the `SendUserMessage` tool and periodic tick mechanism, enabling user-independent continuous workflows
- **Continuous running mechanism**
  - When KAIROS is active and the message queue is idle, `print.ts:2477` automatically injects a proactive tick, allowing the model to "wake up" and continue working without user input — e.g., checking background task status, pushing progress updates
- **Changes to memory mode**
  - Under KAIROS, `buildAssistantDailyLogPrompt` replaces the standard memory prompt: instead of maintaining a MEMORY.md index, the model appends writes to a daily log file (`YYYY/MM/YYYY-MM-DD.md`), which is consolidated into MEMORY.md by a nightly process. This suits long-running sessions, avoiding frequent index file modifications

11.3 Cowork Extra Guidelines Injection

```
// Injected via environment variable
const coworkExtraGuidelines = process.env.CLAUDE_COWORK_MEMORY_EXTRA_GUIDELINES
// Injected into Auto Memory's buildMemoryLines
```

Note:

- Cowork is a desktop/embedded operation mode for Claude Code: **running Claude Code embedded in a desktop application rather than via the terminal CLI**. Users can access it via the Cowork desktop client or Cowork SDK
- **Relationship**: Cowork is the embedder/consumer of Claude Code; Claude Code is the underlying engine layer

Key differences between Cowork and regular CLI

- Message routing is controlled by the desktop application (Cowork bridges WebSocket rather than REPL)
- Memory path is externally controlled (via environment variable injection, not auto-inferred from git root)
- Plugin directory uses `cowork_plugins` instead of `plugins`
- Settings file uses `cowork_settings.json` instead of `settings.json`

Purpose of CLAUDE_COWORK_MEMORY_EXTRA_GUIDELINES
As an embedded mode, Cowork operators (e.g., enterprise IT) need to inject additional behavioral guidelines into the model. These guidelines are environment-level and not hardcoded. In other words, CLAUDE_COWORK_MEMORY_EXTRA_GUIDELINES allows Cowork operators to **inject additional behavioral specifications into the model's memory system at the environment variable level, without modifying Claude Code's own code**.

For example: an enterprise Cowork may require the model to "all code changes must comply with internal coding standards" or "outbound communications must go through specific approval processes". These are injected via this environment variable:

```
const coworkExtraGuidelines = process.env.CLAUDE_COWORK_MEMORY_EXTRA_GUIDELINES
// Injected into the extraGuidelines parameter of the memory prompt
```

It is injected in two places:

- Auto Memory (`loadMemoryPrompt`): guidance for the regular conversation memory system
- Agent Memory (`agentMemory.ts`): guidance for sub-agent memory systems

In `buildMemoryLines` , `extraGuidelines` is appended as a separate paragraph at the end of the memory instructions, so the model sees these additional rules directly when reading.

Purpose of CLAUDE_COWORK_MEMORY_PATH_OVERRIDE
In regular CLI mode, the memory directory is inferred from the git root directory hash, one directory per project. But Cowork's memory path is controlled by external mounting (potentially mounted per user or team scope), so this environment variable directly overrides the path calculation logic, allowing Cowork operators to precisely control memory storage locations.

---

12. Coordinator and Agent Prompt Priority

12.1 Coordinator Mode

Note: **See the "Agent Algorithm Flow" documentation for details**

When `feature('COORDINATOR_MODE') && process.env.CLAUDE_CODE_COORDINATOR_MODE` is set, the Coordinator Prompt is used:

```
// src/utils/systemPrompt.ts
if (feature('COORDINATOR_MODE') && isEnvTruthy(process.env.CLAUDE_CODE_COORDINATOR_MODE)) {
  return asSystemPrompt([getCoordinatorSystemPrompt(), ...appendSystemPrompt])
}
```

12.2 Agent Definition Injection

Built-in Agents (loaded by `loadAgentsDir.ts`) can provide system prompts specific to each agent type:

```
// src/tools/AgentTool/loadAgentsDir.ts
const agentSystemPrompt = mainThreadAgentDefinition
  ? mainThreadAgentDefinition.getSystemPrompt()
  : undefined
```

---

13. Compact (Context Window Compression)

Note: **See the "Context Management" documentation for details**

13.1 Compression Trigger Timing

```
// src/services/compact/autoCompact.ts
// Automatically triggers compression when context approaches token limit
// After triggering, old messages are replaced with summaries via summarizeMessages
```

13.2 Memory Handling During Compression

```
// src/services/compact/sessionMemoryCompact.ts
// Session Memory also participates in the summarization process during compression
// extractMemories service reloads memories after compression
(resetGetMemoryFilesCache('compact'))
```

13.3 Compact Boundary Markers

```
// src/utils/messages.ts
export function createCompactBoundaryMessage(
  partialCompactDirection?: PartialCompactDirection
): SystemCompactBoundaryMessage
// Inserts boundary markers into the compressed message sequence for subsequent tracking
```

---

14. Architecture Summary and Key Design Patterns

14.1 Layered Responsibility Overview

Layer	Mechanism	Trigger Timing	Persistent
System Reminder	`<system-reminder>` XML tag	Always (via message attachments)	No
CLAUDE.md	File loading	Session start + on-demand	No (files themselves are persistent)
Auto Memory	Guidance + memory files	Session start + extraction	Yes (disk files)
Session Memory	Forked subagent	Post-Sampling	Yes (session-level disk file)
ExtractMemories	Forked subagent	StopHooks	Yes (Auto Memory)
Hooks	Shell/callback	Lifecycle events	Varies by hook
Coordinator/Agent	Prompt replacement/append	CLI flag / feature	No
Compact	Summary replacement	Token threshold	No

14.2 Key Design Patterns

Pattern 1: Forked Agent (Isolated Execution)

All potentially side-effect-producing background operations (Session Memory, ExtractMemories) execute via `runForkedAgent()` with isolated context, without polluting main conversation state:

```
// src/utils/forkedAgent.ts
await runForkedAgent({
  promptMessages,
  cacheSafeParams,        // Inherit parent session's prompt cache
  canUseTool,              // Restrict tool permissions
  overrides: { readFileState: isolatedContext.readFileState }
})
```

Pattern 2: GrowthBook Feature Gate

Nearly all new mechanisms have GrowthBook feature flags for controlled rollout:

```
if (!getFeatureValue_CACHED_MAY_BE_STALE('tengu_passport_quail', false)) return
if (!getFeatureValue_CACHED_MAY_BE_STALE('tengu_session_memory', false)) return
```

Note:

- **Definition**: GrowthBook is an A/B testing / feature flag platform that Claude Code uses for controlled rollout and remote control of code behavior, without redeployment
- **Two usage patterns**
  - Build-time constant folding (`feature('FLAG_NAME')`): these flags are used for major product-line switches (KAIROS, BRIDGE_MODE, PROACTIVE, etc.), determining at build time whether to include certain code
  - Runtime remote evaluation (`getFeatureValue_CACHED_MAY_BE_STALE`): the values of these flags are fetched from the GrowthBook server and can be changed at any time, used for: 1) enabling a small feature per user/organization; 2) real-time shutdown of a live feature (kill-switch); 3) A/B testing different parameter values (thresholds, timeouts)
- **User attribute targeting**: GrowthBook determines each flag's return value based on user attributes (user type, subscription tier, org ID, etc.), enabling per-user-group control

Pattern 3: Memoize Caching

Large amounts of context data are cached via `lodash-es/memoize` to avoid redundant computation:

```
export const getSystemContext = memoize(async () => { ... })
export const getUserContext = memoize(async () => { ... })
export const getMemoryFiles = memoize(async () => { ... })
```

Pattern 4: Cache Invalidation Propagation

When values that affect context change, all related caches are cascade-cleared:

```
export function setSystemPromptInjection(value: string | null): void {
  systemPromptInjection = value
  getUserContext.cache.clear?.()
  getSystemContext.cache.clear?.()
}
```

Pattern 5: System Prompt Segmented Caching

`systemPromptSection()` supports a `cacheBreak: boolean` flag, allowing some segments to be recomputed every turn without breaking the entire prompt cache:

```
// Stable segment: cacheBreak: false (cacheable)
export function systemPromptSection(name: string, compute: ComputeFn): SystemPromptSection

// Dynamic segment: cacheBreak: true (recomputed every turn, will break prompt cache)
export function DANGEROUS_uncachedSystemPromptSection(
  name: string,
  compute: ComputeFn,
  _reason: string,  // Must provide reason
): SystemPromptSection
```

Pattern 6: Timing-Aware Extract Mem

ExtractMemories only works when the main Agent does **not** produce memory writes (mutual exclusion), avoiding competition for writes to the same file.

Pattern 7: Memory Type Classification Constraints

Mandatory classification via frontmatter `type` field, with detailed save/access/trust guidelines, preventing the memory system from being abused as general-purpose storage.

Appendix: Key File Index

File	Responsibility
`src/utils/messages.ts`	System Reminder wrapping, message normalization, compact boundary
`src/context.ts`	getSystemContext / getUserContext (git status, CLAUDE.md, date)
`src/utils/claudemd.ts`	Complete CLAUDE.md discovery, loading, parsing, caching pipeline
`src/memdir/memdir.ts`	Auto Memory prompt building, entrypoint truncation
`src/memdir/memoryTypes.ts`	Memory type classification, save/access/trust guidance text
`src/services/SessionMemory/sessionMemory.ts`	Session memory automatic extraction (Post-Sampling)
`src/services/extractMemories/extractMemories.ts`	Conversation history auto-extraction to persistent memory (StopHooks)
`src/utils/hooks.ts`	Complete Hook system (16+ event types)
`src/utils/hooks/postSamplingHooks.ts`	Post-sampling Hook mechanism
`src/constants/prompts.ts`	System Prompt section building
`src/constants/systemPromptSections.ts`	Segmented cache management
`src/utils/systemPrompt.ts`	Effective System Prompt priority building
`src/utils/queryContext.ts`	API cache key prefix building
`src/services/compact/compact.ts`	Context compression
`src/services/compact/autoCompact.ts`	Automatic compression triggering

File	Responsibility
`src/utils/dynamicConfig.ts` / `src/hooks/useDynamicConfig.ts`	Dynamic configuration (GrowthBook)